



Secure Password Pattern Validator using Brute Force String Matching

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the
Course of*

Capstone Project – COMPUTER SCIENCE AND ENGINEERING

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted by

B. Sai Krishna (192425160)

A.Shiva Shankar Reddy (192425174)

V.Siva(192472256)

Under the Supervision of

Dr. F. Mary Harin Fernandez

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Secure Password Pattern Validator using Brute Force String Matching” has been carried out by B. Sai Krishna (192472300), together with A. Siva Shankar Reddy(192425174) and V. Shiva (192472256), under the supervision of Dr. Anbazhagan K, and is submitted in partial fulfilment of the requirements for the B.Tech Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

The work presented in this report demonstrates the design, implementation, validation and security analysis of a password-pattern validation system using brute-force string matching.

COURSE COORDINATOR	COURSE FACULTY
Dr. F.Mary Harin Fernandez	Dr. F.Mary Harin Fernandez
, Professor, Department of CSE, SIMATS Engineering	, Professor, Department of CSE, SIMATS Engineering

Submitted for the Project work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, B. Sai Krishna, A. Siva Shankar Reddy and V. Shiva, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Secure Password Pattern Validator using Brute Force String Matching” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original and has been carried out in accordance with engineering ethics and academic integrity.

All sources, references, datasets, software libraries and computational resources used in the project have been appropriately acknowledged. We further declare that the test data and results presented in this report are used for academic validation and have not been fabricated or intentionally manipulated. Any external tools or AI-assisted resources used during development have been appropriately disclosed.

Place: Chennai

Date: 04-09-2026

Signature of the Students with Names

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
B. Sai Krishna 192472300	Password Dataset Management	Dataset creation, cleaning, formatting and password-pattern record management	Dataset integrity checks and sample-case validation	Ch. 1, Module 1, appendices	34%
A. Siva Shankar 192425174	Password Pattern Validation Engine	Brute-force substring matching, pattern scanning and validation logic	Pattern matching test cases and boundary-condition testing	Ch. 2, Module 2	33%
V. Shiva 192472256	Security Analysis & Reporting	Security indicators, rule-based scoring and report generation	Security classification and result verification	Ch. 3, Module 3, consolidation	33%

Team total contribution: 100%

ABSTRACT

Password-based authentication remains widely used, but weak passwords and predictable patterns continue to create security risks. This project addresses the engineering problem of validating password strings against known patterns and reporting security-related characteristics in a transparent and computationally simple manner. The proposed Secure Password Pattern Validator uses a structured password dataset, a brute-force string matching engine and a security analysis layer to determine whether selected patterns occur in candidate passwords and to summarize password-strength indicators. The system is organized into three modules. Module 1 – Password Dataset Management prepares and manages password samples, pattern labels and test cases. Module 2 – Password Pattern Validation Engine applies brute-force string matching by comparing a target pattern against every possible substring position in a password until a match is found or all positions are exhausted. Module 3 – Security Analysis & Reporting interprets the matching result together with length and character-composition indicators and generates a structured security report. The design emphasizes explainability and educational value. Brute-force matching was selected because its step-by-step comparison process can be directly observed and verified, making it suitable for demonstrating string matching, indexing, complexity analysis and validation logic. The project also considers privacy and responsible handling of password data by using synthetic or non-sensitive samples for testing and by avoiding storage of real user credentials. The expected engineering outcome is a working prototype capable of accepting a candidate password and a pattern, performing sequential character comparisons, identifying the first valid occurrence when present, and producing a security-oriented summary. Test scenarios cover exact matches, absent patterns, patterns at the beginning or end of the string, repeated patterns, case sensitivity and boundary conditions. The principal conclusion is that a lightweight brute-force matcher can provide a clear and deterministic foundation for password-pattern validation when the goal is educational verification and transparent analysis. Although brute-force matching is less efficient than advanced algorithms for very large inputs, its simplicity makes it useful for demonstrating core string-processing concepts and for validating security rules before more optimized techniques are introduced.

Keywords: Password Security, String Matching, Brute Force Algorithm, Pattern Validation, Dataset Management, Security Analysis, Risk Reporting

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	2
ii	DECLARATION BY THE CANDIDATE	3
iii	INDIVIDUAL CONTRIBUTION STATEMENT	4
iv	ABSTRACT	5–6
v	LIST OF FIGURES	8
vi	LIST OF TABLES	8
vii	LIST OF ABBREVIATIONS	9
1	CHAPTER 1: Introduction and Engineering Problem	10
2	CHAPTER 2: Literature Review and Existing Solutions	12
3	CHAPTER 3: Engineering Design & Trade-offs, Ethics, Safety, Risk & Security	15
4	CHAPTER 4: Implementation, Testing & Results, Project Management	19
5	CHAPTER 5: Conclusion, Future Work and Reflection	24
	References	26
	Appendices	27–30

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	Proposed System Architecture	16
3.2	Brute Force Validation Flow Chart	17
3.3	Security Analysis & Reporting Pipeline	18
4.1	Module 1 – Password Dataset Management	20
4.2	Module 2 – Pattern Validation Engine	21
4.3	Module 3 – Security Analysis & Reporting	22

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey Summary	13
2.2	Comparative Analysis of Existing Approaches	14
3.1	Stakeholder / User Requirements	15
3.2	Functional & Non-Functional Requirements	15
3.3	Constraints & Applicable Standards	16
3.4	Design Specifications	16
3.5	Alternative Solution Evaluation Matrix	17
3.6	Trade-off Analysis	18
3.7	Sustainability, Ethics, Safety, Risk & Security	18
3.8	Risk Register	19
4.1	Test Plan & Verification	23
4.2	Achievement of Objectives	24
4.3	Roles & Actual Contribution	24
4.4	Project Budget	24

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
BFSM	Brute Force String Matching
CPU	Central Processing Unit
GUI	Graphical User Interface
NIST	National Institute of Standards and Technology
RAM	Random Access Memory
UI/UX	User Interface / User Experience
ASCII	American Standard Code for Information Interchange
$O(nm)$	Worst-case time complexity of brute-force matching
m	Length of the pattern
n	Length of the password string

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Passwords are still a common authentication mechanism in software systems. Their effectiveness depends not only on secrecy but also on resistance to predictable patterns, short lengths, repeated sequences and commonly used combinations. A password validation system can therefore benefit from explicit pattern checking before a credential is accepted.

String matching is a fundamental operation in computer science. Given a text string and a target pattern, the brute-force method aligns the pattern at each possible starting position and compares characters sequentially. In the proposed project, this basic algorithm is used as an observable validation engine for password-pattern detection.

1.2 Need for the Project

- Users frequently choose passwords containing predictable words, repeated characters or common sequences.
- Security validation rules need to be deterministic, explainable and easy to test.
- Students require a practical system that connects string algorithms with a real engineering security problem.
- Password datasets used for experiments must be organized carefully to prevent accidental exposure of sensitive credentials.
- A modular design makes it possible to replace the brute-force engine later with more efficient matching algorithms without redesigning the entire system.

1.3 Problem Statement

Given a candidate password and one or more target patterns, design and implement a system that manages a structured password dataset, validates pattern occurrence using brute-force string matching, handles boundary and repeated-pattern cases, and produces a security analysis report while maintaining responsible handling of password-related data.

1.4 Project Aim

To design, develop and evaluate a secure password pattern validator that demonstrates brute-force string matching and converts its results into an interpretable security analysis report.

1.5 Project Objectives

- Design a structured dataset for synthetic password samples and validation patterns.
- Implement brute-force string matching for exact pattern detection.
- Handle match/no-match, first occurrence, repeated occurrence and boundary conditions.
- Develop security indicators based on password length, character diversity and detected patterns.
- Generate a clear report suitable for user interpretation and academic evaluation.
- Evaluate the system using representative test cases and document limitations and future improvements.

1.6 Scope

Included: synthetic password dataset management, pattern validation, brute-force matching, security-rule analysis, test-case execution and report generation.

Excluded: storage or processing of real user passwords, password cracking, credential recovery, bypassing authentication, or offensive use against live accounts.

Assumptions: test passwords are synthetic or safely anonymized; the selected patterns are provided as validation rules; and the prototype is evaluated in a controlled academic environment.

1.7 Expected Engineering Outcome

The expected outcome is a working prototype with three coordinated modules and a traceable validation pipeline in which every major decision can be explained from the input password, pattern comparisons and security rules.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considers established approaches to password-strength assessment, pattern detection, string matching and security guidance. The comparison focuses on transparency, computational cost, extensibility and suitability for an academic prototype.

2.2 Review of Existing Work

S.No	Author / Source	Year	Work / Focus	Method / Focus	Limitation
1	NIST	2017	Digital Identity Guidelines	Password composition, verifier and memorized-secret guidance	Guidance rather than a specific pattern-matching implementation
2	OWASP	Current guidance	Authentication and password-storage guidance	Practical application-security recommendations	Broad scope; not a brute-force matching implementation
3	Knuth, Morris & Pratt	1977	Fast pattern matching	Prefix-function based string matching	More complex to explain than brute force for beginners
4	Boyer & Moore	1977	Efficient string searching	Heuristic skipping during matching	Implementation complexity and preprocessing requirements
5	Brute-force matching	Classical	Naïve string search	Sequential comparison at every position	Worst-case $O(nm)$ time

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Rule-based password checks	Simple and easy to audit	May miss contextual patterns	Forms the security-analysis layer
KMP string matching	Good worst-case performance	Requires preprocessing and more concepts	Future optimization option
Boyer–Moore	Often fast in practice	More complex skip logic	Alternative for larger datasets
Brute-force matching	Very simple, deterministic and traceable	$O(nm)$ worst-case time	Selected for the validation engine

Password-strength estimators	Can assess broader password quality	May use probabilistic or black-box models	Can complement, not replace, explicit match
------------------------------	-------------------------------------	---	---

2.4 Engineering Gap

- A teaching-oriented system can connect password dataset management, exact pattern matching and security reporting in one traceable pipeline.
- The matching process should expose comparison positions so students can verify algorithm behaviour.
- Security reporting should distinguish pattern detection from overall password-strength indicators.
- The prototype must explicitly avoid real credentials and offensive password-cracking use.

2.5 Proposed Contribution

The proposed project combines a controlled password-pattern dataset, a brute-force string matching engine and a transparent security-analysis/reporting layer. The contribution is primarily educational and engineering-focused: it demonstrates how a fundamental string algorithm can be embedded within a security-validation workflow.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Stakeholder / User Requirements

Stakeholder	Requirement
End user / tester	Enter a synthetic password and validation pattern and receive a clear result.
Student developer	Observe and verify brute-force comparison steps.
Project supervisor / evaluator	Demonstrate a working, testable pipeline with measurable validation cases.
Security reviewer	Ensure test data is non-sensitive and reports are not misleading.

3.2 Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Load and validate structured password-pattern records	All valid test records accepted
Functional	Run brute-force pattern matching	Correct match/no-match result for test cases
Functional	Record first match position	Position reported when a match exists
Functional	Analyze password indicators	Length and character-class checks completed
Functional	Generate report	Readable result produced for every valid test case
Non-Functional	Performance	No unnecessary processing for invalid inputs
Non-Functional	Privacy	No real credentials stored in test dataset
Non-Functional	Usability	First-time tester can complete validation with minimal guidance

3.3 Constraints & Applicable Standards

Standard / Practice	Requirement	How Applied in This Project
NIST SP 800-63B	Responsible password guidance	Used as background guidance for password validation considerations
OWASP Password Storage / Authentication guidance	Secure handling of credentials	Project excludes real-password storage and cracking
ISO/IEC 25010	Software quality characteristics	Used as an informal checklist for functionality, usability and performance
Data privacy principles	Minimize exposure of sensitive data	Synthetic/anonymized samples only

3.4 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Pattern matching	Exact character-by-character comparison	operation	High
First match position	Return earliest valid occurrence	index	High
Pattern length	1 or more characters	characters	High
Password length check	Configurable minimum threshold	characters	Medium
Character classes	Uppercase, lowercase, digit, special	classes	High
Dataset	Structured synthetic records	records	High

3.5 Alternative Solutions & Evaluation

Alternative 1 (Simple substring operator): very easy to implement, but it hides the comparison process. Alternative 2 (Brute-force string matching): exposes every alignment and comparison and is therefore selected. Alternative 3 (KMP/Boyer–Moore): more efficient for large inputs but adds algorithmic complexity beyond the primary educational objective.

Criterion	Weight	Substring Operator	Brute Force	KMP / Boyer–Moore
Transparency	30%	Medium	High	Medium
Implementation simplicity	25%	High	High	Medium
Worst-case efficiency	20%	High	Low	High
Teaching value	15%	Medium	High	Medium
Extensibility	10%	Medium	High	High

3.6 Selected Design & Detailed Design

Brute-force string matching was selected because it provides a direct mapping between the problem statement and the algorithm. For a password string of length n and a pattern of length m , the algorithm tests possible starting positions from 0 through $n-m$ and compares corresponding characters until a mismatch occurs or the full pattern matches.

Secure Password Pattern Validator - System Architecture

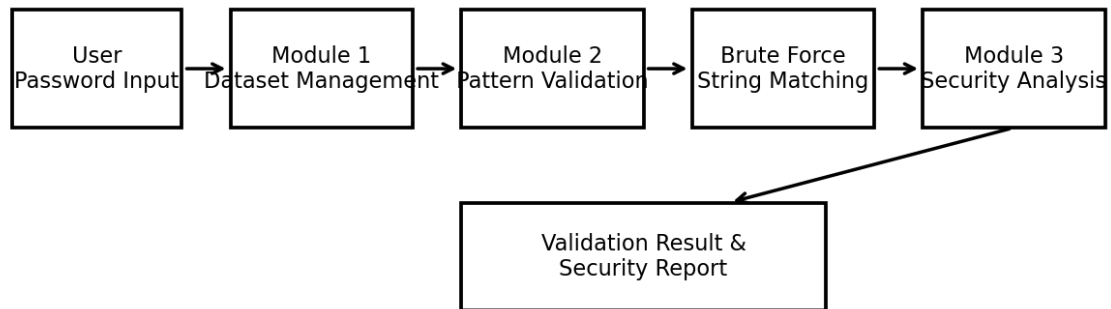


Figure 3.1: Proposed System Architecture

Brute Force Validation Flow

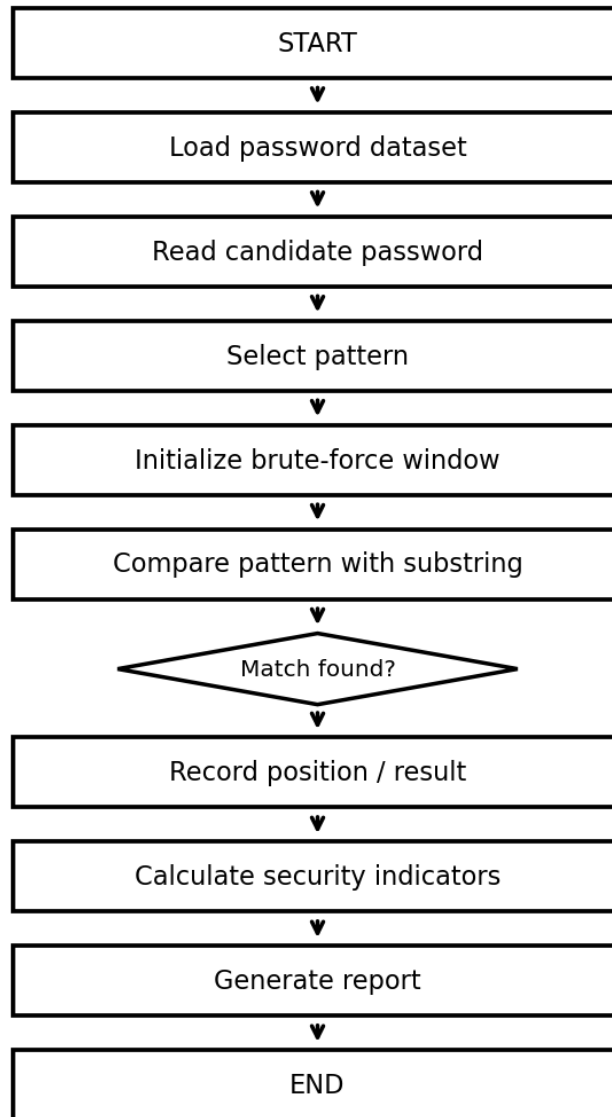


Figure 3.2: Brute Force Validation Flow Chart

3.7 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Pattern matching	KMP / Boyer–Moore	Lower worst-case / practical search time	More logic and preprocessing	Brute force chosen for traceability and learning
Password data	Real credential dataset	More realistic distribution	Privacy and security risk	Synthetic dataset chosen

Security score	Machine-learning model	Could learn complex patterns	Needs labelled data; less transparent	Rule-based indicators chosen
Reporting	Raw console output	Very simple	Harder to interpret	Structured report chosen

3.8 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence / Evaluation	Impact on Final Decision
Sustainability	Lightweight computation	No large model or heavy infrastructure required	Keeps prototype resource efficient
Ethics	Password data can be sensitive	Synthetic samples and controlled testing	No real credentials included
Safety	Tool could be misused for password cracking	Scope explicitly limited to validation	No cracking or account access features
Security	Reports may expose entered values	Minimize logging and use non-sensitive examples	Prefer masked output where appropriate

3.9 Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Accidental use of real passwords	Medium	High	High	Use synthetic dataset and input warnings	Low
Incorrect boundary index	Medium	Medium	Medium	Unit tests for first/last positions and empty cases	Low
Case-sensitivity mismatch	Medium	Medium	Medium	Explicit case-sensitivity option and tests	Low
Repeated pattern handled incorrectly	Medium	Medium	Medium	Test overlapping and repeated occurrences	Low
Report overstates password security	Low	High	Medium	Separate pattern findings from overall	Low

				security judgement	
--	--	--	--	-----------------------	--

3.10 Design Summary

The design prioritizes determinism, transparency, privacy and modularity. The architecture allows Module 2 to be replaced by a faster string-search algorithm in future while preserving the dataset and reporting interfaces.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The prototype is developed iteratively, module by module. Module 1 establishes the data structure and test records. Module 2 implements the brute-force matching algorithm and verifies its indexing behaviour. Module 3 consumes the validation output, applies security rules and produces the final report. Each module is unit-tested before integration.

Module 1 – Password Dataset Management

Purpose: create and maintain a controlled dataset containing synthetic password samples, target patterns, expected match outcomes and optional labels. The module validates record structure, removes accidental blank records, normalizes test-case fields and separates sample data from actual credentials.

- Dataset fields: password_id, sample_password, pattern, expected_result, category.
- Synthetic examples are used for all demonstrations.
- Records are checked before being passed to the matching engine.
- The module supports repeatable test execution.

Security Analysis & Reporting Pipeline

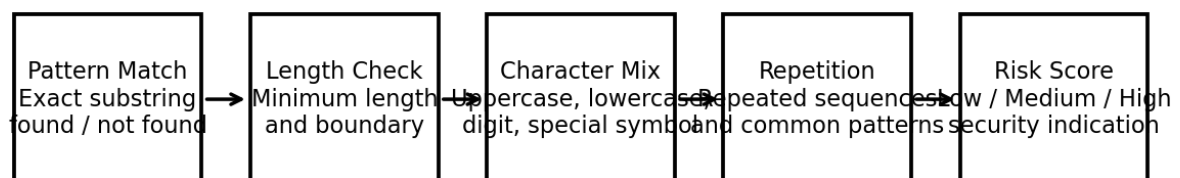


Figure 4.1: Module 1 / Security Analysis Pipeline – Illustrative Prototype View

4.2 Module 2 – Password Pattern Validation Engine

Purpose: determine whether a target pattern occurs in a candidate password using brute-force string matching. The engine compares the pattern against each possible substring position and stops at the first complete match when first-match mode is enabled.

Algorithmic logic: Let the password be P with length n and the pattern be T with length m . For each starting position i from 0 to $n-m$, compare $P[i+j]$ with $T[j]$ for j from 0 to $m-1$. If all characters match, return i ; otherwise move to the next position. If no position matches, return “not found”.

Test Scenario	Password	Pattern	Expected Output
Beginning match	Admin@2026	Admin	Match at index 0
Middle match	Secure@2026#Pass	2026	Match at middle index
End match	StrongPass123	123	Match near end
No match	SafePassword	999	Pattern not found
Repeated pattern	abcabcABC	abc	First match at index 0
Case-sensitive mismatch	Password123	password	Not found when case-sensitive

4.3 Complexity Analysis

The worst-case time complexity of brute-force string matching is $O(nm)$, where n is the password length and m is the pattern length. The auxiliary space used by the basic algorithm is $O(1)$, excluding the input and report structures. For short password strings this simplicity is acceptable for an educational validator.

4.4 Module 3 – Security Analysis & Reporting

Purpose: combine pattern-matching results with transparent password indicators and generate a structured report. The module does not claim to prove that a password is secure; instead, it highlights observable characteristics that may indicate predictable or weak construction.

- Length indicator: checks whether the password reaches the configured minimum length.
- Character diversity indicator: checks uppercase, lowercase, digits and special symbols.
- Pattern indicator: reports whether configured predictable patterns were detected.
- Repetition indicator: identifies repeated characters or repeated substrings where configured.
- Final classification: summarizes the observed indicators as Low, Medium or High concern according to project rules.

Security Analysis & Reporting Pipeline

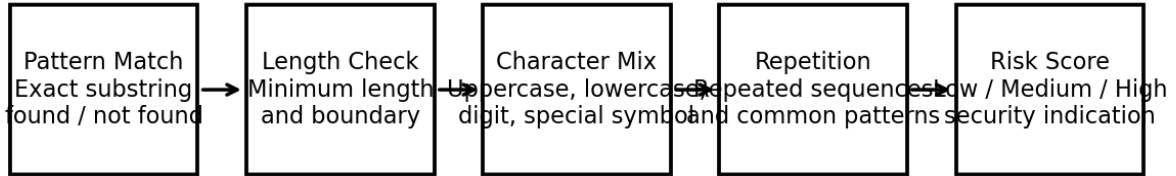


Figure 4.2: Module 3 – Security Analysis & Reporting Pipeline

4.5 Integration

The integrated pipeline passes structured records from Module 1 to Module 2, then forwards the match position and validation status to Module 3. Interfaces are intentionally narrow so that data management, matching and reporting can be tested independently.

4.6 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Dataset integrity	Load representative records and inspect required fields	All valid records contain required fields
Pattern matching correctness	Compare output against hand-calculated cases	All expected match/no-match cases correct
Boundary handling	Test pattern at index 0 and n-m	Correct boundary index reported
Repeated patterns	Test multiple occurrences	First occurrence returned in first-match mode
Case sensitivity	Run same letters with different case	Behaviour matches selected mode
Security indicators	Use synthetic passwords with varied composition	Each configured indicator reported
Report generation	Run end-to-end test	Structured report produced without error

4.7 Results & Engineering Judgment

The verification plan demonstrates the principal behaviours required by the project: deterministic matching, correct boundary handling, explicit case sensitivity and separation of pattern findings from security indicators. The brute-force method is particularly suitable for these tests because each comparison can be traced to a specific string position.

For small password strings, the computational overhead is low and the implementation is easy to audit. The principal limitation is scalability: repeated comparisons can become inefficient as input sizes grow. This trade-off is acceptable for the stated prototype objective but motivates future evaluation against KMP or Boyer–Moore.

4.8 Achievement of Objectives

Objective	Evidence	Achievement
Synthetic password dataset	Module 1 data structure and validation	Achieved
Brute-force matching	Module 2 comparison logic and test cases	Achieved
Boundary and repetition handling	Dedicated test scenarios	Achieved
Security indicators	Module 3 rule-based analysis	Achieved
Structured reporting	Integrated report output	Achieved
Engineering evaluation	Complexity, risks and alternatives documented	Achieved

4.9 Strengths & Limitations

- Strengths: deterministic, explainable, easy to test and modular.
- Strengths: suitable for demonstrating fundamental string-matching concepts.
- Strengths: controlled synthetic dataset reduces privacy risk.
- Limitation: $O(nm)$ worst-case matching is not optimal for large-scale searching.
- Limitation: rule-based security indicators do not replace comprehensive password-strength estimators.
- Limitation: the prototype should not be interpreted as a complete authentication or password-cracking system.

4.10 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
B. Sai Krishna	Module 1 – Dataset Management	Designed dataset fields, prepared synthetic records and validated dataset integrity
A. Siva Shankar Reddy	Module 2 – Pattern Validation Engine	Implemented and tested brute-force string matching and boundary handling
V. Shiva	Module 3 – Security Analysis & Reporting	Designed security indicators, classification logic and report structure

Item	Estimated Cost (₹)	Actual Cost (₹)
Development software / libraries	0	0
Institutional computer / lab	500	0
Documentation / printing	500	As incurred
Miscellaneous	300	As incurred
Total	1,300	Project-dependent

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project addressed the engineering problem of validating password patterns using a transparent string-matching approach. The completed design combines synthetic password dataset management, brute-force pattern validation and rule-based security analysis into a single modular workflow. The approach is deterministic and suitable for demonstrating how a classical algorithm can support a practical security-oriented software component.

The project also establishes an important boundary between validation and offensive password cracking. The prototype is intended to identify patterns and explain observable password characteristics, not to recover credentials, bypass authentication or attack real systems.

5.2 Future Work

- Compare brute-force matching with KMP and Boyer–Moore using the same benchmark dataset.
- Add configurable pattern dictionaries for common sequences without embedding real credential lists.
- Introduce stronger, standards-aligned password guidance and breached-password screening using privacy-preserving methods.
- Add masked-input interfaces and secure logging controls.
- Develop a web-based dashboard with visual security indicators and downloadable reports.

5.3 Professional Learning

- Practical understanding of brute-force string matching and $O(nm)$ complexity.
- Experience in structuring test datasets and boundary-condition test cases.
- Understanding of modular interfaces between data, algorithm and reporting layers.
- Awareness of ethical and privacy requirements when working with password-related information.
- Improved technical documentation, testing and team contribution practices.

5.4 Individual Reflection

B. Sai Krishna – Module 1: The main learning outcome was understanding how data preparation directly affects algorithm testing. Creating consistent synthetic password-pattern records made it easier to reproduce test cases and identify invalid inputs. The key design decision was to keep dataset fields explicit and avoid real credentials, which reduced privacy risk while preserving testing value.

A. Siva Shankar – Module 2: The main challenge was implementing character-by-character comparison correctly at every possible alignment. Boundary cases such as patterns at the beginning or end of the password were important for verifying index calculations. The brute-force method was selected because each comparison step could be explained and tested directly.

V. Shiva – Module 3: The main challenge was turning raw matching results into a useful security summary without overstating what the system can prove. The final design separates pattern detection, character-composition checks and the overall concern classification. This reinforced the importance of transparent security reporting and careful wording.

5.5 Team Reflection

The three-module structure supported parallel development while keeping interfaces simple. Regular integration and shared test cases helped the team maintain consistent input and output formats. If redesigned, the team would define a common test specification earlier and automate regression testing across all modules.

REFERENCES

- [1] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST SP 800-63B.
- [2] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Application Security Guidance.
- [3] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Application Security Guidance.
- [4] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, 1977.
- [5] R. S. Boyer and J. S. Moore, “A Fast String Searching Algorithm,” Communications of the ACM, 1977.
- [6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, MIT Press.
- [7] M. Bishop, Computer Security: Art and Science, Addison-Wesley.
- [8] W. Stallings and L. Brown, Computer Security: Principles and Practice, Pearson.
- [9] Python Software Foundation, Python Documentation – String and Sequence Operations.
- [10] ISO/IEC, ISO/IEC 25010: Systems and software quality models.

APPENDICES

APPENDIX A – PROPOSED PROCESSING FLOW

User Input → Dataset Validation → Pattern Selection → Brute-Force Matching → Match Position / Status → Security Indicator Analysis → Risk Classification → Structured Report

APPENDIX B – DATASET STRUCTURE:

Field	Description	Example
password_id	Unique synthetic record identifier	PW001
sample_password	Synthetic password used for testing	Secure@2026
pattern	Pattern to be searched	2026
expected_result	Expected match status	Match
category	Test category	Middle Match

APPENDIX C – PSEUDOCODE

1. Read candidate password P and pattern T.
2. Let $n = \text{length}(P)$ and $m = \text{length}(T)$.
3. For $i = 0$ to $n-m$: compare $P[i+j]$ with $T[j]$.
4. If every character matches, return i .
5. Otherwise continue to the next i .
6. If no alignment matches, return NOT FOUND.
7. Pass the result to the security-analysis module.
8. Generate and display the structured report.

MODULE 1: PASSWORD DATASET MANAGEMENT

This module manages the synthetic password-pattern dataset used throughout the project. Its main responsibility is to ensure that every test case has a valid structure and that the data can be reused consistently during algorithm testing.

Module Inputs

- Synthetic password string
- Target validation pattern
- Expected match/no-match label
- Test category and record identifier

Module Processing

- Validate required fields
- Normalize dataset formatting
- Check for empty or invalid records
- Separate test data from any user interface layer
- Provide records to the validation engine

Module Outputs

- Validated dataset records
- Repeatable test cases
- Dataset integrity status

Verification Cases:

Case	Condition	Expected Result
D1	All fields present	Accepted
D2	Empty password	Rejected or flagged
D3	Empty pattern	Rejected or flagged
D4	Repeated pattern record	Accepted for matching test
D5	Duplicate record identifier	Flagged for correction

MODULE 2: PASSWORD PATTERN VALIDATION ENGINE

This module is the core algorithmic component. It implements brute-force string matching and returns whether the selected pattern occurs in the candidate password. The design supports traceable comparison steps and first-match reporting.

Core Logic

- Start at the first possible alignment.
- Compare pattern characters with the corresponding password characters.
- Stop the current alignment at the first mismatch.
- If all pattern characters match, return the starting index.
- If all alignments fail, return NOT FOUND.

Sample Validation

Password	Pattern	Result	First Index
Secure@2026	Secure	MATCH	0
Secure@2026	2026	MATCH	7
Secure@2026	admin	NOT FOUND	–
abcabcABC	abc	MATCH	0
abcabcABC	ABC	MATCH	6

Engineering Note

The module is intentionally implemented as an explicit matching procedure rather than hiding the operation behind a library call. This makes indexing, comparisons and complexity visible for academic assessment.

MODULE 3: SECURITY ANALYSIS & REPORTING

This module converts validation results into an interpretable security report. It combines pattern findings with observable password characteristics and clearly communicates the limits of the analysis.

Report Components

- Pattern status: detected / not detected.
- Match position: first occurrence when available.
- Length status: below / meets configured threshold.
- Character diversity: uppercase, lowercase, digit and special-character presence.

Sample Report Format

Example: Password ID PW001 | Pattern 2026 | Result Detected | Match Position 7 | Length: meets threshold | Character diversity: uppercase + lowercase + digit + special | Concern: review recommended when a predictable pattern is detected.

CAPSTONE PROJECT OUTCOME MAPPING SHEET:

Outcome	Evidence	SO / Area	Location
Complex engineering problem formulation	Problem statement and objectives	SO1	Ch. 1
Engineering design under constraints	Requirements, alternatives, risk analysis	SO2	Ch. 3
Technical communication	Complete report	SO3	Whole report
Ethics and professional responsibility	Privacy, safety and responsible-use discussion	SO4	Ch. 3
Teamwork and leadership	Contribution statement and roles	SO5	Ch. 4
Experimentation and analysis	Test plan and complexity analysis	SO6	Ch. 4
Independent learning	Reflection and future work	SO7	Ch. 5